

Principal component analysis

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
from sklearn.decomposition import PCA

plt.style.use("ggplot")
```

In this notebook, we exemplify **principal component analysis** (PCA) by applying it to a data set of images of hand-written digits, which we load now. The data set is provided with the book Hastie, Tibshirani and Freedman: *Elements of Statistical Learning*, source: <https://hastie.su.domains/ElemStatLearn/data.html>. For convenience, the data are provided in the archive **zip_data.zip**, the contents of which must reside in the same directory as this notebook.

```
In [2]: zip_train = np.loadtxt("zip.train")
```

The object `zip_train` is a 7291×257 matrix where each row corresponds to an individual greyscale image, such that the first column denotes the label (0 to 9) of the image and the remaining 256 columns represent $16 \times 16 = 256$ pixels flattened to a vector with values in $[-1, 1]$ (-1 is white; 1 is black).

```
In [3]: print(zip_train.shape)
print(zip_train[:,1])

(7291, 257)
[[ 6.   -1.   -1.   ... -1.   -1.   -1.   ]
 [ 5.   -1.   -1.   ... -0.671 -0.828 -1.   ]
 [ 4.   -1.   -1.   ... -1.   -1.   -1.   ]
 [ 7.   -1.   -1.   ... -1.   -1.   -1.   ]
 [ 3.   -1.   -1.   ... -0.883 -1.   -1.   ]
 [ 6.   -1.   -1.   ... -1.   -1.   -1.   ]]
```

We extract the subset of all images depicting the digit 3. After that, we discard the column consisting of labels.

```
In [4]: threes = zip_train[zip_train[:, 0] == 3]
threes = threes[:, 1:]
```

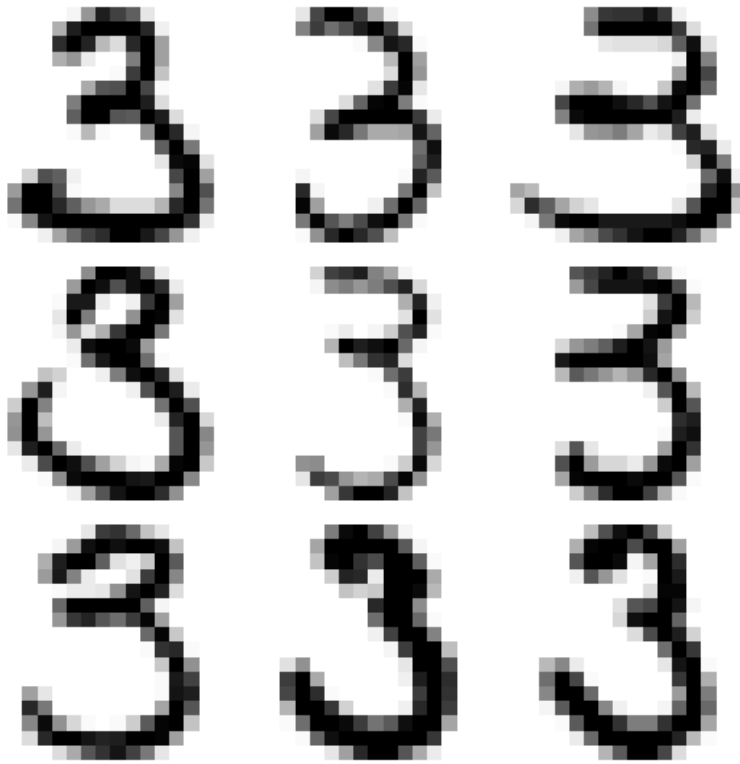
To verify that we have indeed extracted images of 3's, we plot the first nine images.

```
In [5]: def draw_img(ax, x):
    img = (1 - x) / 2
    img = img.reshape(16, 16)
    ax.imshow(img, cmap="gray", vmin=0, vmax=1)
    ax.axis("off")

fig, axes = plt.subplots(3, 3, figsize=(5, 5))

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes[i,:])

plt.tight_layout()
plt.show()
```



To perform PCA we create an object of class `PCA` from `sklearn.decomposition` and call method `fit` :

```
In [6]: threes_pca = PCA()  
        threes_pca.fit(threes);
```

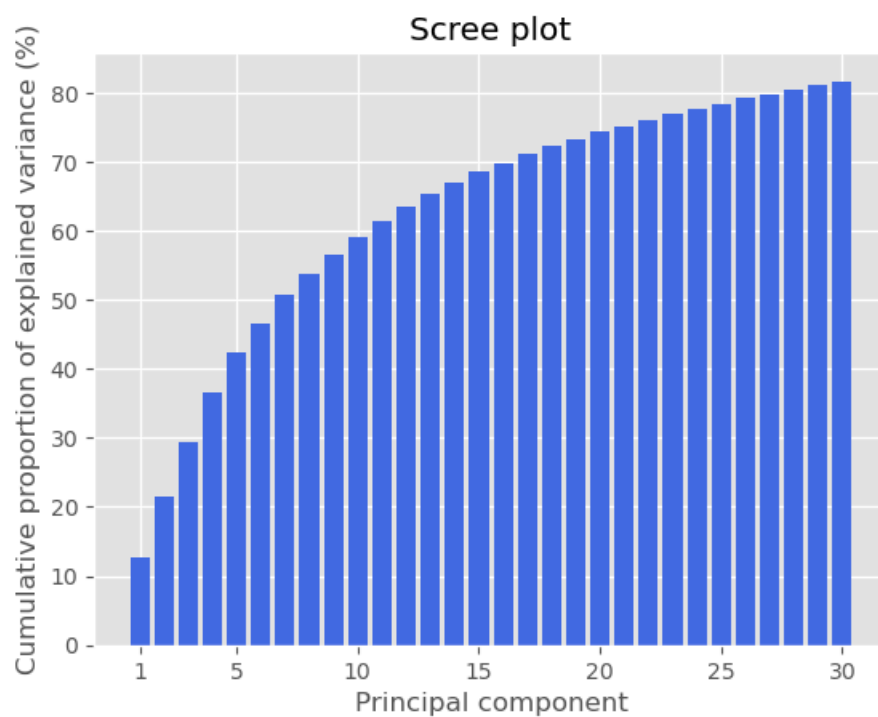
The explained variance ratios are stored in the attribute `explained_variance_ratio_`, using which we can draw a scree plot:

```
In [7]: num_pcs = 30  
        plt.bar(  
            range(1, num_pcs + 1),  
            threes_pca.explained_variance_ratio_[:num_pcs] * 100,  
            color="salmon",  
            zorder=2,  
        )  
        plt.xticks([1, 5, 10, 15, 20, 25, 30])  
        plt.title("Scree plot")  
        plt.xlabel("Principal component")  
        plt.ylabel("Proportion of explained variance (%)")  
        plt.show()
```



Or cumulatively:

```
In [8]: num_pcs = 30
plt.bar(
    range(1, num_pcs + 1),
    np.cumsum(threes_pca.explained_variance_ratio_[:num_pcs]) * 100,
    color="royalblue",
    zorder=2,
)
plt.xticks([1, 5, 10, 15, 20, 25, 30])
plt.title("Scree plot")
plt.xlabel("Principal component")
plt.ylabel("Cumulative proportion of explained variance (%)")
plt.show()
```



We will now experiment with dimensionality reduction using the data. The argument `n_components` of the constructor of `PCA` makes it possible to constrain the number of retained PCs. Subsequently, the

`fit_transform` method outputs the retained PCs. Let us first try an extreme setting where we keep just **one PC**.

```
In [9]: threes_pca_red = PCA(n_components=1)
        threes_reduced = threes_pca_red.fit_transform(threes)
```

The method `inverse_transform` performs reconstruction:

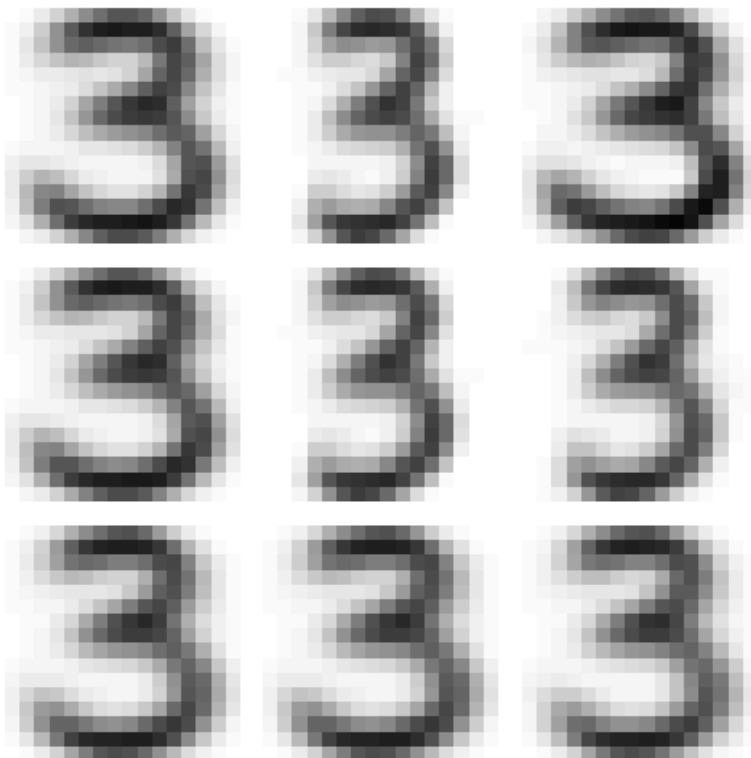
```
In [10]: threes_recons = threes_pca_red.inverse_transform(threes_reduced)
```

Let us then plot (some of) the reconstructed images:

```
In [11]: fig, axes = plt.subplots(3, 3, figsize=(5, 5))

        for i, ax in enumerate(axes.flat):
            draw_img(ax, threes_recons[i,])

        plt.tight_layout()
        plt.show()
```



The results are not too bad. The images are a bit blurry but convey the essential visual information surprisingly well. It should be pointed out, however, that they represent just small deviations from the sample mean, which looks like this:

```
In [12]: draw_img(plt, np.mean(threes, axis=0))
        plt.show()
```



Let us then try **ten PCs**:

```
In [13]: threes_pca_red = PCA(n_components=10)
threes_recons = threes_pca_red.inverse_transform(threes_pca_red.fit_transform(threes))

fig, axes = plt.subplots(3, 3, figsize=(5, 5))

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes_recons[i,])

plt.tight_layout()
plt.show()
```



We get more detail, and more individual properties of each image are recovered. But at the same time, there are also some artifacts around the digits. If we then try **fifty PCs**, we recover even more details:

```
In [14]: threes_pca_red = PCA(n_components=50)
threes_recons = threes_pca_red.inverse_transform(threes_pca_red.fit_transform(threes))

fig, axes = plt.subplots(3, 3, figsize=(5, 5))
```

```

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes_recons[i,])

plt.tight_layout()
plt.show()

```



Finally, we verify that if we include **all PCs**, then we fully recover the original images:

```

In [15]: threes_recons = threes_pca.inverse_transform(threes_pca.fit_transform(threes))

fig, axes = plt.subplots(3, 3, figsize=(5, 5))

for i, ax in enumerate(axes.flat):
    draw_img(ax, threes_recons[i,])

plt.tight_layout()
plt.show()

```

3
8
3

3
3
3

3
3
3